

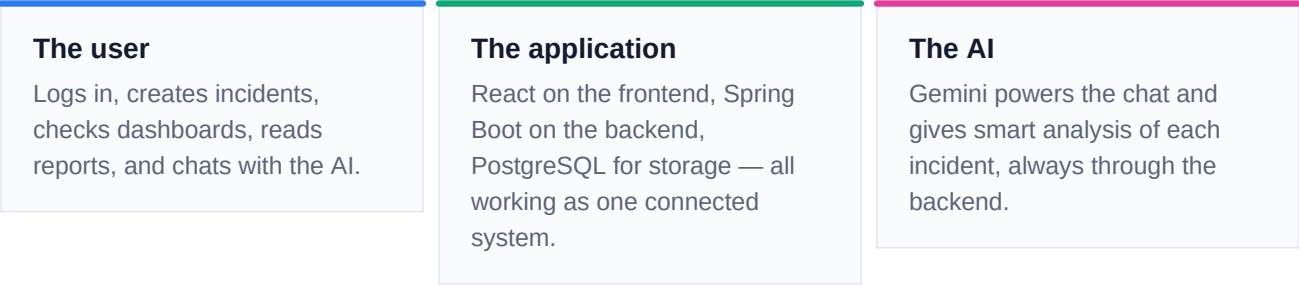
IncidentFlow AI, explained in plain English

A full-stack, AI-powered incident management app for IT and software teams — how every piece fits together, from the login screen down to the database, and how Gemini AI slots into the middle without ever touching the browser.



01 What is IncidentFlow AI?

It's one web app where an IT team can create, track, and resolve “incidents” (problems), while an AI Copilot helps them understand what's going on.

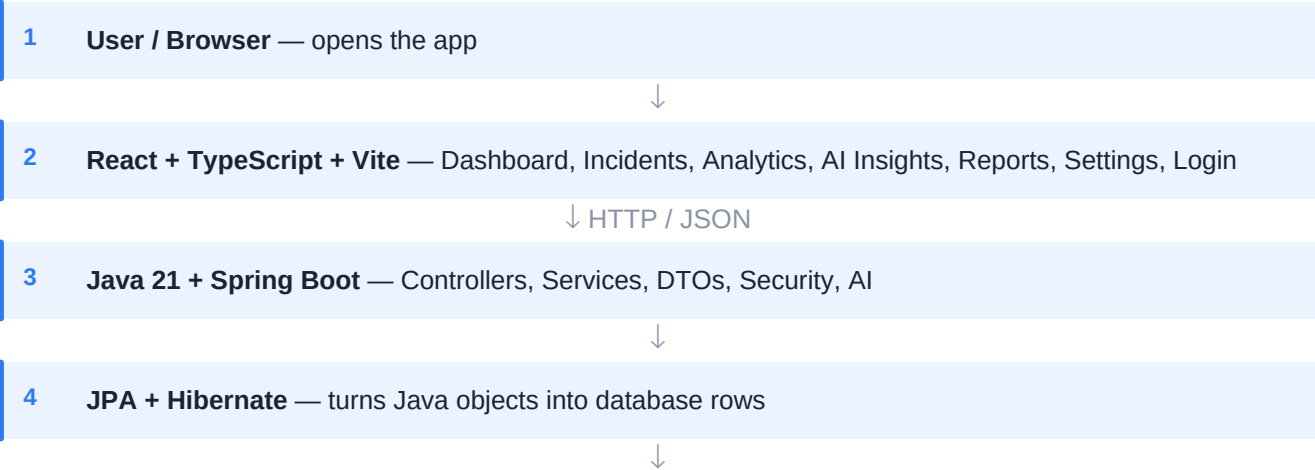


TIP

The backend isn't split into microservices — it's a single Spring Boot app (a “modular monolith”) neatly organized into controllers, services, repositories, DTOs, entities and security code.

02 The big picture: system architecture

This is the one diagram to understand before anything else — everything else in this guide zooms into one piece of it.



5 PostgreSQL 18 — where everything is finally stored

AI

AI path: React → Spring Boot's AI endpoint → Gemini API → back through Spring Boot → React.

Layer	What it's responsible for
Frontend	Shows the screens and captures what the user does.
Controller	Receives HTTP requests and sends back responses.
Service	Holds the actual business logic.
Repository	Reads and writes data that needs to be saved.
Database	Stores the application's data long-term.
AI integration	Connects the backend's AI services to Gemini.

03 Tech stack, in plain words

Every technology used, and the one-line “why” behind it.

Frontend

React builds the interactive screens. **TypeScript** adds safety by typing the JavaScript. **Vite** runs and builds it all fast. **Fetch API** sends requests to the backend.

Backend

Java 21 runs the server code. **Spring Boot** gives it structure and REST APIs. **Spring Security** plus **JWT** handle authentication. **BCrypt** hashes every password.

Data

PostgreSQL 18 stores everything. **Spring Data JPA** and **Hibernate** map Java objects straight onto database tables — no manual SQL wiring.

AI

Gemini API generates chat replies and incident analysis. It's only ever called from the backend, never from the browser.

INFRA

Deployment: Netlify hosts the React build, Render hosts the Spring Boot backend and the PostgreSQL database, and Docker packages the backend for that deployment.

04 Frontend — what the user sees

Screens, forms, navigation, and calling the backend — that's the frontend's whole job.

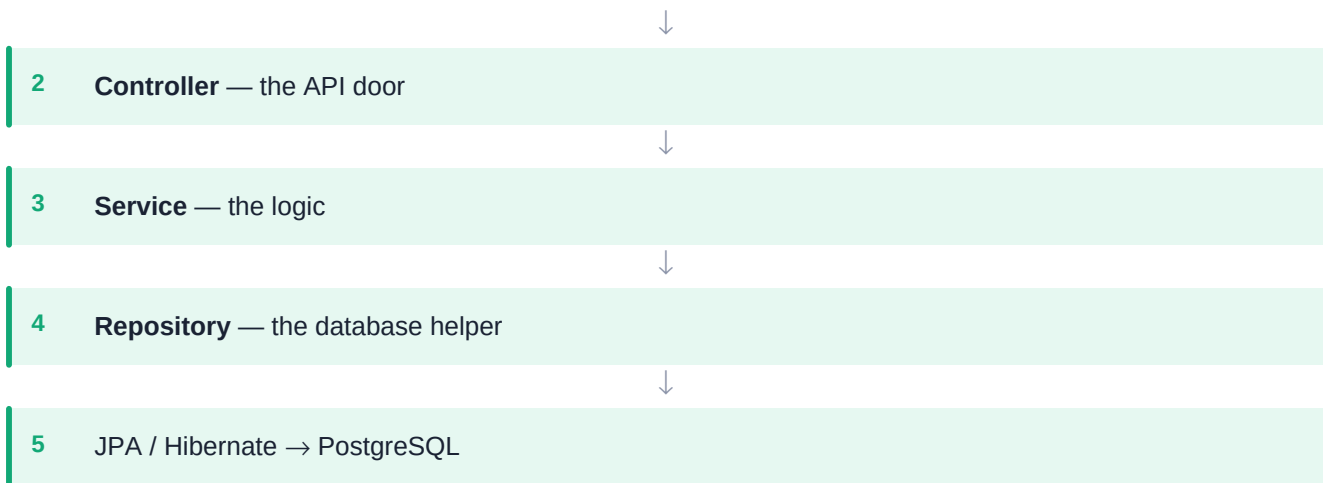
- 1 **App.tsx** — the root of the app
- ↓
- 2 Authentication + navigation state
- ↓
- 3 Dashboard · Incidents · Analytics · AI Insights · Reports · Settings
- ↓
- 4 API service files → Fetch → Spring Boot

The frontend handles the login/register screens, stores the JWT after login, makes protected API calls, and displays incident data, dashboards, AI answers, analytics, reports and settings. The login/register screen also has a 3D animated robot visual and supports five roles: ADMIN, MANAGER, SUPPORT_AGENT, DEVELOPER and CUSTOMER.

05 Backend — the brain

Every request passes through the same chain: door → logic → database helper.

- 1 HTTP Request arrives



Part	Easy meaning
Controller	API entry point — receives HTTP requests.
DTO	Carries request and response data.
Entity	The Java version of a saved database record.
Service	Application and business logic.
Repository	Database access.
Security	JWT authentication and Spring Security.

06 Registration, login & JWT

How a person becomes a logged-in, verified user.



SECURITY

The plain password is never stored — only its BCrypt hash. And the app doesn't keep server-side sessions; it stays “stateless,” relying entirely on the JWT for each request.

07 User roles

ADMIN	MANAGER	SUPPORT_AGENT	DEVELOPER	CUSTOMER
-------	---------	---------------	-----------	----------

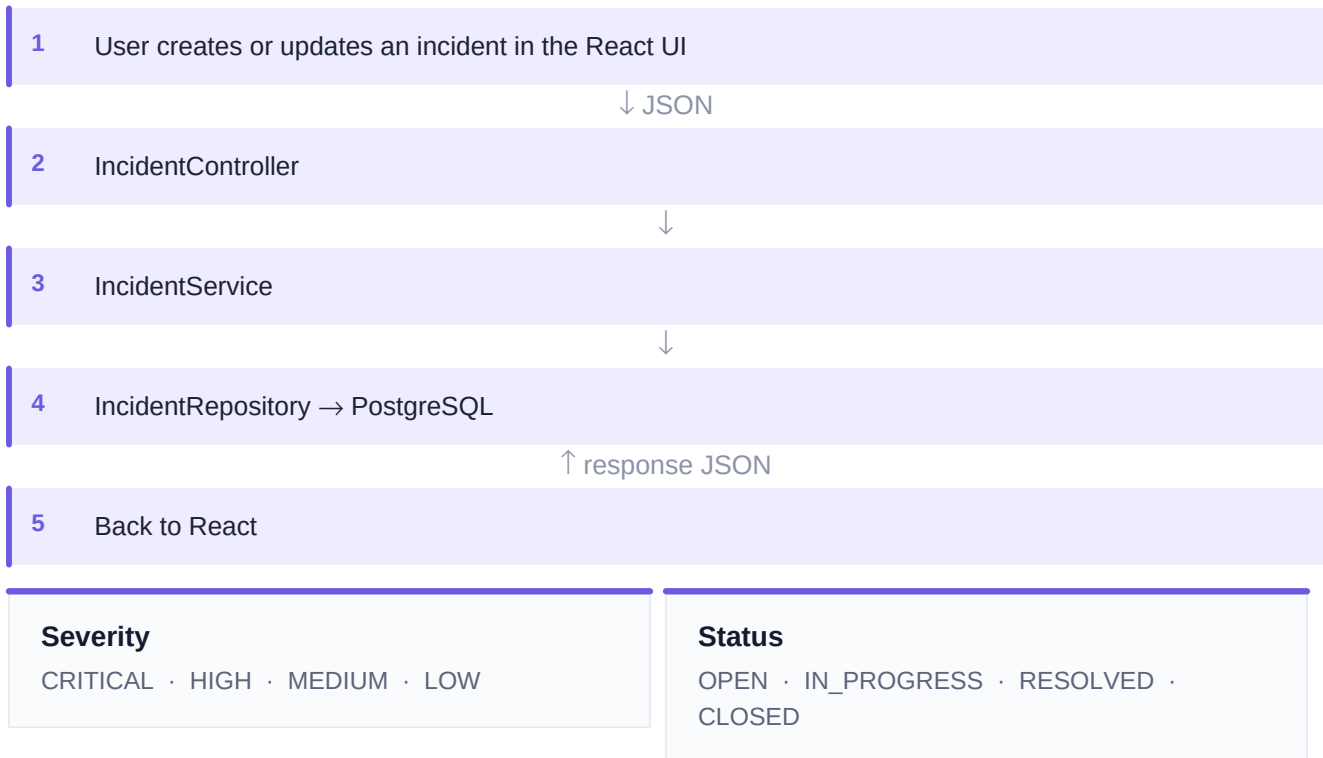
The role lives on the User entity, and Spring Security represents it as an authority with a ROLE_ prefix.

NOTE

Keep this honest in interviews: role-aware login and role storage are real and implemented. Don't claim totally separate screens or permissions per role unless that authorization logic is actually built and tested.

08 Incident management — the core flow

An incident is simply an IT/software problem that needs tracking through to resolution.



Every incident stores a title, description, severity, status, timestamps, and who created it.

09 Dashboard & analytics

Dashboard

KPI-style numbers, recent incidents, trend visuals, and the entry point into AI insights.

Analytics

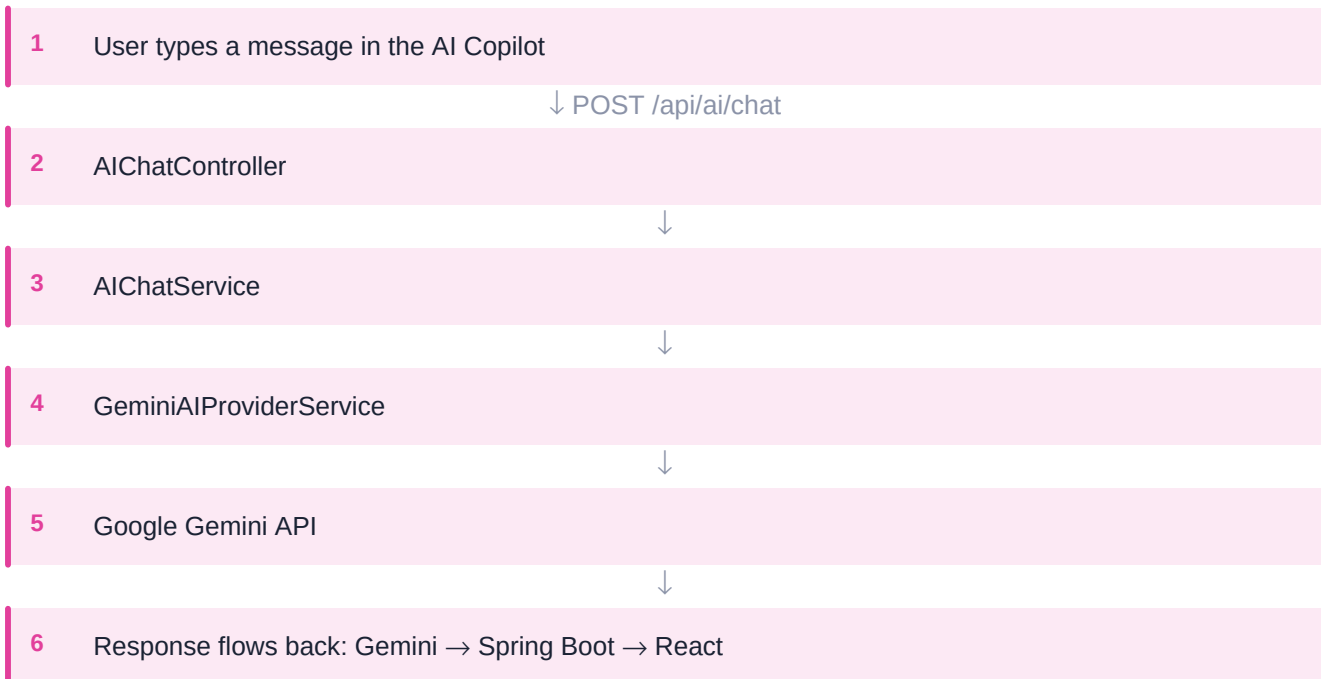
Incident statistics, status breakdowns, severity breakdowns, and general operational analysis.

NOTE

Some comparison labels on screen are visual touches, not real historical measurements — don't describe them as historical data unless they're actually backed by stored history.

10 AI Copilot — the full chat flow

The single most important AI rule in this project: **React never touches the Gemini key.** React only ever talks to Spring Boot.



Keeping Gemini behind the backend keeps the secret key safe on the server. The AI provider layer supports both chat and incident-analysis operations.

11 AI incident analysis



3 AIAnalysisService



4 GeminiAIProviderService → Gemini API



5 Analysis is returned to the UI

There's also a GET `/api/incidents/{id}/ai-analysis` endpoint to retrieve an analysis that was already generated. The prompt is built around the incident's own details and can produce a summary, possible root causes, and recommended next actions.

12 Reports & Settings

Reports

Operational report view, KPI info, status distribution, severity breakdown, resolution health, an AI operations summary, CSV export, and print.

Settings

Profile and preferences, AI Copilot preferences, notifications, automatic refresh, system status, platform info, and saving preferences.

Together these turn the project from basic CRUD into something that combines incident management, analytics, AI, reporting and settings.

13 Database & ORM

PostgreSQL 18 stores the data; Spring Data JPA and Hibernate handle the mapping so nobody writes raw SQL by hand for everyday operations.

1 Java Entity



2 JPA / Hibernate



3 Repository



4 PostgreSQL



5 Data → Service → Controller → React

ORM

ORM = Object-Relational Mapping — Java objects are mapped straight onto relational database rows. User stores id, name, email, password hash and role; Incident stores title, description, severity, status, timestamps and who created it.

14 Important APIs

Endpoint	Purpose
POST /api/auth/register	Create a user account.
POST /api/auth/login	Authenticate and return a JWT.
GET /api/health	Check backend health.

Endpoint	Purpose
/api/incidents	Incident CRUD operations.
POST /api/ai/chat	AI Copilot chat.
POST /api/incidents/{id}/ai-analysis	Generate AI incident analysis.
GET /api/incidents/{id}/ai-analysis	Retrieve a saved AI incident analysis.

15 Deployment architecture

1 User Browser

↓ HTTPS

2 **Netlify** — hosts the production React build

↓ API requests

3 **Render** — hosts the Spring Boot backend

↓ JDBC

4 **Render PostgreSQL** — the application database

The backend's AI calls reach out separately to the Google Gemini API. Docker packages the backend for deployment.

SECRETS

Secrets like DB credentials, the JWT secret, and the Gemini API key live in environment variables — never committed to GitHub.

16 Project structure — the easy map

Not a full file listing — just enough to navigate the project confidently in an interview.

```
backend/
  ■■
  src/main/java/com/incidentflow/incidentflow/
    ■■ controller/
    ■■ dto/
    ■■ entity/
    ■■ repository/
    ■■ security/
    ■■ service/
    ■■ IncidentflowApplication.java
```

```
frontend/
  ■■ src/
    ■■ components/
    ■■ services/
    ■■ App.tsx
    ■■ main.tsx
    ■■ CSS files
```

17 Three flows you must know cold

A) Normal incident flow

User → React → REST API → Controller → Service
→ Repository → PostgreSQL → Response → React

B) Login flow

User → React → /api/auth/login →
AuthenticationManager → user lookup → BCrypt
check → JWT → React → localStorage

C) AI chat flow

User → React Copilot → /api/ai/chat → AIChatController → AIChatService → GeminiAIProviderService → Gemini
→ Response → React

18 Interview cheat sheet

What is your project?

An AI-powered, full-stack incident management platform.

Why React?

For an interactive, component-based frontend.

Why Spring Boot?

For structured REST APIs and clean integration with security, data and AI.

Why PostgreSQL?

For reliable, relational storage of structured application data.

Why JWT?

To authenticate protected requests using a stateless token.

Why BCrypt?

To hash passwords instead of ever storing them in plain text.

What is JPA/Hibernate?

JPA gives the persistence abstractions; Hibernate maps Java entities onto relational data.

How does React talk to Java?

Through HTTP REST APIs, exchanging JSON.

How does Gemini fit in?

React calls the backend, the backend calls Gemini, and the response travels back the same way.

Is it microservices?

No — it's a modular monolith, one Spring Boot app organized into clean layers.

Where is it deployed?

Frontend on Netlify; backend and PostgreSQL on Render.

19 The whole project in one sentence

“IncidentFlow AI is a Java Spring Boot and React full-stack incident management platform where users can securely log in, manage incidents, view analytics and reports, and use Gemini-powered AI through a backend API — with PostgreSQL for storage and Netlify/Render for deployment.”

This guide sticks to what's actually built in IncidentFlow AI today — no unverified future features presented as done.